Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Q.1

a.

$$f(\omega) = \frac{1}{2} \| x\omega - y \|^2$$

Gradient update rule for gradient descent

proof of convergence

learning rate $(\eta) < \frac{2}{\lambda_{max}}$

$$\frac{\partial f}{\partial x} = (x\omega - y)\,\omega$$

$$\frac{\partial f}{\partial y} = -(x\omega - y)$$

$$\frac{\partial^2 f}{\partial x^2} = \omega^2$$

$$\frac{\partial^2 f}{\partial y^2} = 1$$

$$\frac{\partial}{\partial x}\left(\frac{\partial f}{\partial y}\right)$$

$$\frac{\partial}{\partial x}(y - x\omega) = -\omega$$

$$\frac{\partial}{\partial y}\left(\frac{\partial f}{\partial x}\right)$$

$$\frac{\partial}{\partial y}(-\omega)$$

So we get our Hessian matrix as follows.

Sr. No. 6175

START WRITING HERE
(Begin answer for each question on a new page)

Space For Marks	Question No.	
	Q. 1	
	a.	

Hessia Matrix

$$\begin{bmatrix} w^2 & -w \\ -w & 1 \end{bmatrix}$$

Now lets calculate
eiger values and vectors

lets assume w as 2

$$\begin{bmatrix} 4 & -2 \\ -2 & 1 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 4-\lambda & -2 \\ -2 & 1-\lambda \end{bmatrix} = 0$$

$$4-\lambda(1-\lambda) - 4 = 0$$

$$\cancel{4} - \lambda - 4\lambda + \lambda^2 - \cancel{4} = 0$$

$$\lambda^2 - 5\lambda = 0$$
$$\lambda(\lambda - 5) = 0$$

$$\lambda = 0$$
$$\lambda = 5$$

So we can say $\begin{bmatrix} -1 & -2 \\ -2 & -4 \end{bmatrix}$

Both det are 0

so yes

learning rate $\begin{bmatrix} 4 & -2 \\ -2 & 1 \end{bmatrix}$

$$0 < \eta < \frac{2}{\lambda_{max}}$$

Hence prooved

Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q.1 b.	

Bias - variance decomposition

For the expected squared error of a predictor

now take any MSE

Take a regression problem of overfitting

→ why does it happens.

→ As we know overfitting means working well on train dataset and poor on test data set

→ Low Bias and High variance tradeoff leading to a high capacity.

→ This affects generalization whenever we are testing / predicting the new data points

→ So we need to use L1, L2 regularization, Dropout regularization, optimization algorithms like Adam, momentum in gradient descent while backword propagation.

START WRITING HERE
(Begin answer for each question on a new page)

Space For Marks	Question No.	

Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q. 2	
	a.	

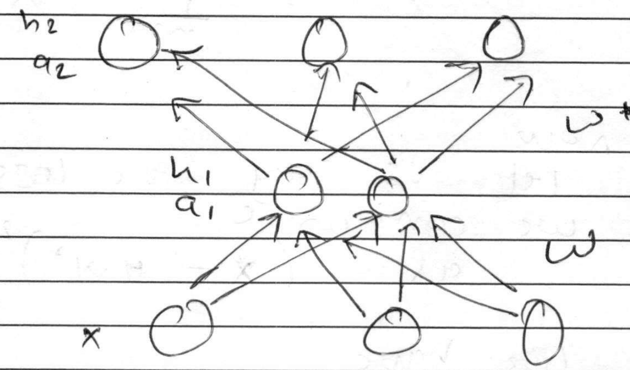
Sr. No. 6175

Space For Marks	Question No.	**START WRITING HERE** (Begin answer for each question on a new page)

Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q.2	
	b.	

An Linear under complete autoencoder trained with MSE is equivalent to PCA.



An undercomple autoencoder
→ It encodes dimensions of hidden layer lesser than that of input layer dim(h) < dim(x)

Requirement
① It should have a linear encoder
② It should have a linear decoder
③ Loss function used is MSE

④ Normalized form

$$x^* = \frac{1}{\sqrt{m}} \underbrace{\left(x - \frac{1}{m} \Sigma x \right)}_{x'}$$

This will ensure data to be ~~cente~~ 0 centered so it will be normalized.

Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Q.2

b.

We can also say

$$X^{*} = \frac{1}{\sqrt{m}} X'$$

$$X^{T} X = \frac{1}{m} X' X'^{T}$$

Now let us say the loss function is we can write as $(X - HW^{*})^2$

We have to prove $HW^{*} = X$

Now let us use SVD

$$H = U \Sigma$$

$$W^{*} = V^{T}$$

$HW^{*} =$

$$(XX^{T})(XX^{T})^{-1} \underbrace{U \Sigma V^{T}}$$

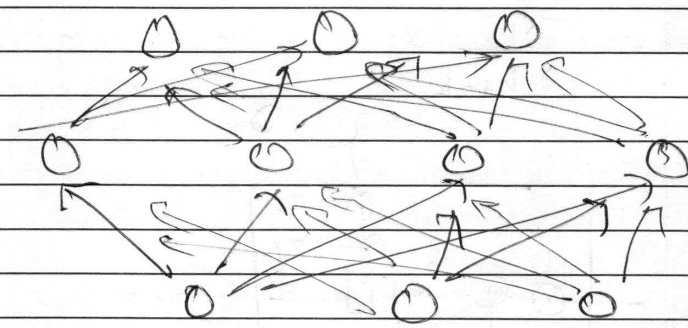
$$\cancel{X} \underbrace{X^{T} (X^{T})^{-1}} \underbrace{X^{-1} X}$$

X.

So we get back our input

That's why we say with certain conditions an undercomplete auto encoder works as PCA.

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q2	
	b.	An overcomplete autoencoder



→ It has neurons more ~~layers~~ in hidden layer as compared from input layer

→ It encodes to a higher dimension of neurons

$dim(h) \geq dim(x)$

and then decodes to the output layer as same dimension as input layer.

→ So there's a very high chance it tends just to copy and paste the identity function.

→ To avoid this we use regularized auto encoders such as

Sparse Autoencoder
Noisey Autoencoder
Contractive Autoencoder.

→ By making some of the neurons inactive at time so it won't just copy ~~ma~~ paste.

→ By various methods mentioned we can achieve our goal.

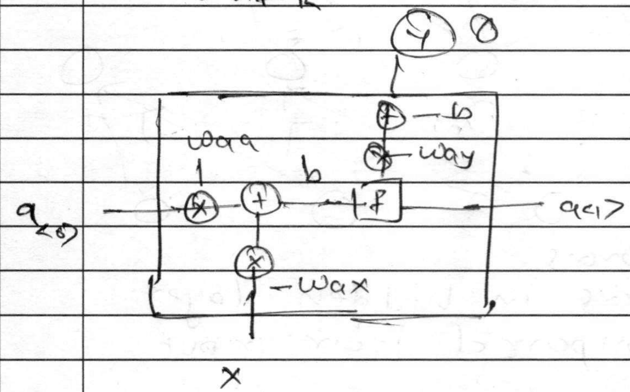
Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Q.3

a. RNN (Recurrent Neural Network)

$\frac{\partial L}{\partial h_{t-k}}$



$$a_t = f\left(W_{aa}\, a_{t-1} + W_{ax}\, x_t + b_a\right)$$

$$O = f\left(W_{ay}\, a_t + b_y\right)$$

Gradient descent expression for RNN.

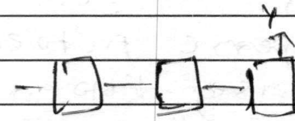
For hidden layers

Lets take two hidden layers

$L \rightarrow O \rightarrow a_3 -$

$\quad \hookrightarrow a_2 -$

$\quad\quad \hookrightarrow a_1$



Just for the last O/p it would

$L \rightarrow O \rightarrow a_3 \rightarrow W_3$

$$\frac{\partial L}{\partial W_3} = \frac{\partial L}{\partial O} \times \frac{\partial O}{\partial a_3} \times \frac{\partial a_3}{\partial W_3}$$

Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q3 a.	

But for hidden layers
as we know W_{aa} and W_{ax} are
used in all 3 cells
so,

$$\frac{\partial L}{\partial W_{aa}} = \frac{\partial L}{\partial O} \times \frac{\partial O}{\partial a_3} \times \frac{\partial a_3}{\partial W_{aa}}$$

$$+ \frac{\partial L}{\partial O} \times \frac{\partial O}{\partial a_3} \times \frac{\partial a_3}{\partial a_2} \times \frac{\partial a_2}{\partial W_{aa}}$$

$$+ \frac{\partial L}{\partial O} * \frac{\partial O}{\partial a_3} \times \frac{\partial a_3}{\partial a_2} \times \frac{\partial a_2}{\partial a_1} \times \frac{\partial a_1}{\partial W_{aa}}$$

same would be
applied for W_{ax}

As we can say that there
are many derivatives for just
3 cells so imagine for a
large layer
and the multiplication will
come out to be close to 0
and this is called vanishing
gradient problem.

So to fix this problem
LSTM comes into picture.
(Long short term memory)

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Q.3

a.

LSTM (Long short Term memory)

$$C_t = (f_t * C_{t-1}) + (i_t \tilde{C}_t)$$

$$\tilde{C}_t = \tanh(W_c(h_{t-1}, x_t) + b_c)$$

$$f_t = \sigma(W_f(h_{t-1}, x_t) + b_f)$$

$$i_t = \sigma(W_i(h_{t-1}, x_t) + b_i)$$

$$O_t = \sigma(W_o(h_{t-1}, x_t) + b_o)$$

and
the final O/P

$$h_t = O_t * \tanh(C_t)$$

Now in LSTM we have forget gate and input gate.

unlike RNN here new values are passed by candidate value and input gate and we can forget and retain information using forget gate.

so for applications like checking grammar for long paragraphs and long texts, we need module like LSTM which can help in retaining the context and remove after some time.

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Q.3

b.

Gating equations of GRU

Its a lightweight version of LSTM

$$\tilde{c}_t = u_t * c_t + (1-u_t) h_{t-1}$$

$$u_t = \sigma(w_u(h_{t-1}, x_t) + b_u)$$

$$r_t = \sigma(w_r(h_{t-1}, x_t] + b_r)$$

$$h_t = \tanh(w_h[r_t * h_{t-1}, x_t] + b_h)$$

We have
only two gates update gate
and reset gate

So we can say that mathematically
GRU have lesser parameters than
LSTM (2 instead of 3 gates)

and this also improves overall speed
and faster computations can be
achieved via it.

So this is the reason we use
GRUs over LSTM
(1) better speed
(2) better generalization
(3) Needs less parameters to be
trained on
(we can say from the above
mathematical equations)

Sr. No. 6175

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
		Rough Work